

Guía de Laboratorio · Sesión 3

Tu primera app multi-contenedor — un blog en red aislada

Curso 101-EAN · Gestión de Contenedores en Entornos de Nube Híbrida · Duración: 1 hora Plataforma: **Play with Docker** (navegador, gratis)

Hoy montas una aplicación de verdad: un blog WordPress con su base de datos MySQL, conectados por nombre en una red que TÚ creas — con la base de datos invisible para el mundo exterior. Entregable: **el blog vivo + la evidencia del aislamiento**.

0. Requisitos (5 min)

Entra a labs.play-with-docker.com → Login → Start → + ADD NEW INSTANCE.

Parte 1 · Redes y el superpoder del DNS (15 min)

1.1 — Mira lo que ya existe:

```
docker network ls          # bridge, host, none: las de fábrica
```

1.2 — Crea TU red:

```
docker network create ean-net
docker network inspect ean-net    # tu subred, aún sin contenedores
```

1.3 — La prueba del nombre. Dos contenedores en TU red:

```
docker run -d --name uno --network ean-net alpine sleep 3600
docker run -d --name dos --network ean-net alpine sleep 3600
docker exec uno ping -c 2 dos      # ✅ se encuentra POR NOMBRE
```

1.4 — El contraste. Ahora dos en la red de fábrica:

```
docker run -d --name tres alpine sleep 3600
docker run -d --name cuatro alpine sleep 3600
docker exec tres ping -c 2 cuatro  # ❌ bad address: sin DNS en bridge
```

👉 **Para la bitácora:** ¿por qué `uno` encuentra a `dos` pero `tres` no encuentra a `cuatro`? ¿Qué te da la red propia que la de fábrica no?

1.5 — Limpieza parcial: `docker rm -f uno dos tres cuatro`

Parte 2 · El blog completo (30 min)

2.1 — La base de datos (fíjate: SIN `-p` — decisión de seguridad):

```
docker run -d --name db --network ean-net \
-e MYSQL_ROOT_PASSWORD=Ean2026root \
-e MYSQL_DATABASE=wordpress \
-e MYSQL_USER=wp \
-e MYSQL_PASSWORD=Ean2026wp \
mysql:8.0
```

MySQL tarda ~30-60 s en inicializar. Míralo con `docker logs -f db` — cuando diga `ready for connections` (la segunda vez), está listo. (Ctrl+C para salir del log.)

2.2 — El blog (con `-p`: esta Sí es la puerta de entrada):

```
docker run -d --name blog --network ean-net -p 8080:80 \
-e WORDPRESS_DB_HOST=db \
-e WORDPRESS_DB_USER=wp \
-e WORDPRESS_DB_PASSWORD=Ean2026wp \
-e WORDPRESS_DB_NAME=wordpress \
wordpress:6
```

Nota la línea clave: `WORDPRESS_DB_HOST=db` — **el nombre**, no una IP.

2.3 — Instálalo y publica tu primer post: clic en el enlace **8080** → asistente de WordPress (idioma → título del sitio → usuario/contraseña del blog) → **Entradas** → **Añadir nueva** → escribe algo y publícalo.

2.4 — Diagnostica como profesional:

```
docker network inspect ean-net    # el plano: blog y db, sus IPs internas
docker exec blog ping -c 2 db    # el blog alcanza a la db por nombre
docker logs blog | tail -5       # el blog sano
```

👉 **Para la bitácora:** captura del blog con tu post + salida del `network inspect`. ¿Qué IP interna tiene la db? ¿Importa? ¿Por qué no?

Parte 3 • Comprueba el aislamiento (15 min)

3.1 — La db no tiene puerta. Compara:

```
docker ps --format 'table {{.Names}}\t{{.Ports}}'
```

El blog muestra `0.0.0.0:8080->80`; la db solo `3306/tcp` (sin flecha = sin puerta). En Play with Docker no aparece ningún enlace para el 3306: **no hay forma de llegarle desde afuera.**

3.2 — El intruso. Un contenedor en OTRA red no ve a la db:

```
docker network create otra-red
docker run -d --name intruso --network otra-red alpine sleep 3600
docker exec intruso ping -c 2 db      # ❌ bad address: la red lo aísla
```

3.3 — Pero el blog sí (misma red):

```
docker exec blog ping -c 2 db      # ✅
```

Eso es segmentación: **quién puede hablar con quién lo decide la red, no la suerte.**

👉 **Para la bitácora (entregable):** 1. Dibuja tu arquitectura: las dos redes, los tres contenedores, la única puerta (8080). 2. ¿Por qué a Colpensiones le importa que la BD no tenga puerto publicado? 3. ¿Qué pasaría si mañana necesitas un segundo blog? ¿Misma red o red nueva? ¿Por qué?

3.4 — NO borres el blog todavía — haz la prueba final de la sesión:

```
docker rm -f blog
docker run -d --name blog --network ean-net -p 8080:80 \
-e WORDPRESS_DB_HOST=db -e WORDPRESS_DB_USER=wp \
-e WORDPRESS_DB_PASSWORD=Ean2026wp -e WORDPRESS_DB_NAME=wordpress \
wordpress:6
```

Recarga el 8080: **tu post sigue ahí** (vive en la db, no en el blog). Ahora la pregunta incómoda... ¿y si muere la **db**? 🤖 Esa es la Sesión 4.

3.5 — Limpieza final: `docker rm -f blog db intruso && docker network rm ean-net otra-red`

Checklist de la sesión

- ☐ Creé mi red y comprobé el DNS por nombre (y su ausencia en bridge).
- ☐ Levanté MySQL sin puerto publicado y WordPress con `-p 8080:80`.
- ☐ Instalé el blog y publiqué un post.

- ☐ Diagnostiqué con network inspect, ping y logs.
- ☐ Demostré el aislamiento con el contenedor intruso.
- ☐ Vi que el post sobrevive al reemplazo del blog (pero... ¿y la db?).

Entregable de la S3

1. Captura del **blog con tu post**.
 2. Evidencia del aislamiento: el **ping** del intruso fallando vs. el del blog funcionando.
 3. **Diagrama de tu red** (a mano o digital) + las 3 reflexiones en la bitácora.
-

Próxima sesión: la pregunta que quedó sembrada — los datos. Volúmenes, persistencia y respaldo: que la información sobreviva a cualquier contenedor. 🗄️